Массивы, vector, string

Введение в ПИ (C++)

Arrays, Vectors, Strings

Фиксированный массив

```cpp
int a[5] = {10, 20, 30, 40, 50};
std::cout << a[0] << " " << a[4] << "\n";
// a[5] --- out of bounds: undefined behavior
```

Размер известен на этапе компиляции (для обычного локального массива).

Индексация



Индексы с **0**. Последний элемент: `size - 1`.

std::vector — динамический массив

```cpp
#include <vector>

std::vector<int> v = {1, 2, 3};
v.push_back(4);
std::cout << v.size() << " " << v[0] << "\n";
```

Растёт по мере необходимости; хранит элементы подряд в памяти.

Обход

```
for (std::size_t i = 0; i < v.size(); ++i) {
    std::cout << v[i] << ' ';
}

for (int x : v) {              // range-for
    std::cout << x << ' ';
}
```

operator[] vs at

- `v[i]` — без проверки границ (быстрее, опаснее при ошибке)
- `v.at(i)` — бросает исключение при выходе за границы

На старте курса: сами следите за индексами; `at` полезен при отладке.

std::string

```cpp
#include <string>

std::string s = "hello";
s += ", world";
std::cout << s.size() << " " << s[1] << "\n";   // e
if (s.find("world") != std::string::npos) {
    std::cout << "found\n";
}
```

C-строка vs `std::string`

- `"text"` — строковый литерал (`const char[]`)
- `std::string` — удобный класс: длина, конкатенация, поиск
- В новом коде курса предпочитайте `std::string`

Частые ошибки границ

- Цикл до `<= size` вместо `< size`
- Пустой `vector`/`string`: `v[0]` недопустим
- Сравнение C-строк через `==` указателей — не содержимое

Викторина 1 — индекс

Викторина

Дан `std::vector<int> v = \{7, 8, 9\};`. Что такое `v[3]`?

1. `0`
2. `9`
3. выход за границы (ошибка / UB для `[]`)
4. `3`

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 1

Ответ

3. Валидные индексы: 0, 1, 2. `v[3]` — за границей.

Викторина 2 — размер после push

Викторина

После `v.push\_back(4)` у `\{1,2,3\}` чему равен `v.size()`?

1. 3
2. 4
3. 5
4. зависит от capacity

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 2

Ответ

2. `size()` — число элементов (4). `capacity` может быть $\geq$ size.